

Real-time lens distortion algorithm on embedded GPU systems

Young-Woo Kim

Korea University of Technology and Education
(KOREATECH)

Cheon-an, South Korea

aister9@koreatech.ac.kr

Duksu Kim

Korea University of Technology and Education
(KOREATECH)

Cheon-an, South Korea

bluekds@koreatech.ac.kr

ABSTRACT

The lens distortion is essential for displaying VR contents on a head-mounted display (HMD) with a distorted display surface. We propose a novel lens distortion algorithm on an embedded GPU system. To minimize the memory access overhead, we propose a compressed form of a lookup table. We also utilize the integrated memory architecture of the edge GPU system (e.g., NVIDIA's Jetson devices) to reduce data communication overhead between host and device. As a result, our method shows up to 1.72-times higher performance than prior lookup table-based lens distortion approaches while it consumes up to 28.93% less power. Finally, our algorithm achieved real-time performance for high-resolution images on edge GPU systems (e.g., 94 FPS for 8K image on Jetson NX). These results demonstrate the benefits of our approach from the perspectives of both performance and energy.

CCS CONCEPTS

• **Computer systems organization** → **Embedded systems**; • **Computing methodologies** → **Parallel algorithms**.

KEYWORDS

Head Mounted Display, Stereoscopic, Distortion, GPU, Edge device

ACM Reference Format:

Young-Woo Kim and Duksu Kim. 2022. Real-time lens distortion algorithm on embedded GPU systems. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Posters (SIGGRAPH '22 Posters)*, August 07-11, 2022. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3532719.3543241>

1 INTRODUCTION

A head-mounted display (HMD) is the most widely employed device for virtual reality. To provide a reasonable field-of-view, the HMD's optical lens system locates the images at a comfortable distance from user eyes and it magnifies the image. Since such optical lens system causes non-linear distortion on the image, it requires a pre-distortion process to correct the distortion [Robinett and Rolland 1992].

Since the lens system in a device is fixed, we can pre-compute and store the results of the distortion equation for each pixel in a lookup table. Then, we can perform the distortion process at run-time by

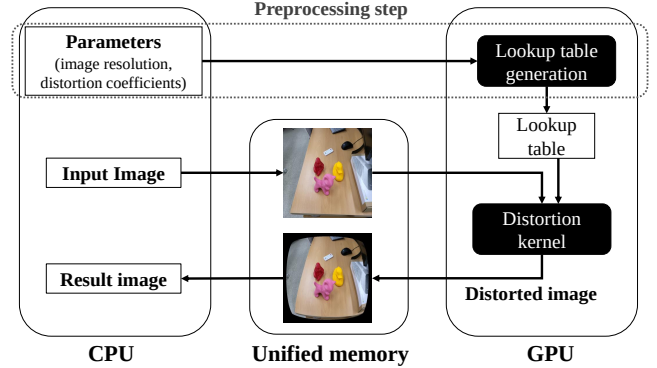


Figure 1: Overview of the proposed system

simply reading the lookup table. This lookup table method is one of the most efficient and widely employed approaches for accelerating the lens distortion process [Qureshi 2013]. Also, there have been approaches to using parallel computing hardware, including multicore-CPU [Daloukas et al. 2010], GPU [Van der Jeught et al. 2012], and FPGA [Bell et al. 2009]. Shehrzad Qureshi [Qureshi 2013] presented a GPU implementation strategy for the lookup table method.

Recent edge devices have powerful multi-core CPU cores and embedded GPU. Using such a small but powerful edge device efficiently is key to improving the convenience of using HMDs. However, prior GPU-based algorithms are less optimized for the architecture of edge GPU devices.

We propose a novel lens distortion algorithm that fits edge devices having an embedded GPU. Our method reduces the stall by memory latency up to 31.12% with a compressed lookup table. Also, it minimizes the data transfer overhead between CPU and GPU memories by exploiting the integrated memory architecture of edge GPU devices. As a result, it shows up to 1.72-times higher performance and takes up to 28.93% less power than the prior lookup table-based algorithm. For example, our algorithm achieved up to 94 FPS and consumed 65.15 mW/frame for an 8K image on a Jetson NX device, while the prior lookup table algorithm showed 55 FPS and took 90.88 mW/frame.

2 OUR APPROACH

A prior implementation of the lookup table-based lens distortion algorithm uses two separate tables for x- and y-dimensions [Qureshi 2013]. The lookup table approach is a memory-bounded work since its primary work is to read the lookup table. To minimize the memory access overhead for the lookup table, we compress the two separate tables as a single table by encoding x- and y-dimension

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH '22 Posters, August 07-11, 2022, Vancouver, BC, Canada

© 2022 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-9361-4/22/08.

<https://doi.org/10.1145/3532719.3543241>

Table 1: System specification of two edge devices in our experiments. The asterisk(*) means the system shared memory. The bold font in the power row marks the corresponding power mode we used for the experiments.

	Jetson Nano	Jetson NX
CPU architecture	ARM	ARM
# of cores (Clock)	4 (1.43 GHz)	6 (1.4 GHz)
GPU architecture	Nvidia Maxwell	Nvidia Volta
# of GPU cores	128	384
Memory size	4 GB*	8 GB*
Memory type	LPDDR4	LPDDR4x
Memory bandwidth	25.6 GB/s	51.2 GB/s
Power	5W / 10W	10W / 15W
Size	70×45 mm	103×90.5 mm

information into a value. In detail, we use the image resolution as a key depending on the property that the one axis (height or width) is longer than (or equal to) another axis. Eq. 1 and Eq. 2 are our encoding and decoding equations where (x, y) is the input coordinates, $c_{x,y}$ is the compressed value.

$$c_{x,y} = \begin{cases} x * width + y, & \text{if } width \geq height \\ y * height + x, & \text{otherwise} \end{cases} \quad (1)$$

$$(x, y) = \begin{cases} (c_{x,y}/width, c_{x,y} \bmod width), & \text{if } width \geq height \\ (c_{x,y} \bmod height, c_{x,y}/height), & \text{otherwise} \end{cases} \quad (2)$$

We found that our compressed lookup table method halves memory load operations, decreasing stalls on lens distortion kernel by memory operation up to 31.12%.

Our algorithm aims to run on edge devices with an embedded GPU. Embedded GPU systems usually have integrated memory architecture while CPU and GPU share the memory space. Exploiting such an architectural advantage, we use a unified memory space [Cheng et al. 2014] to minimize the data transfer overhead between CPU and GPU. Fig. 1 shows the overview of the proposed lens distortion system.

3 RESULTS

We implemented our lens distortion algorithm and an alternative based on Qureshi’s method [Qureshi 2013] (i.e., GPU_{LUT}). GPU_{LUT} uses two separate lookup tables for x- and y-dimensions and does not use unified memory. Then we tested the performance and energy efficiency of two algorithms on two edge GPU systems (Table 1).

Fig. 2 shows the performance of GPU_{LUT} and ours. Our method generally shows better performance for each device with each resolution. Our method shows up to 1.26 and 1.71 times higher performance over GPU_{LUT} on Jetson Nano and Jetson NX, respectively. This result validates the efficiency of our compressed lookup table over the two separate lookup tables used in GPU_{LUT} .

Fig. 3 shows the power consumption of GPU_{LUT} and our approach. Our compressed lookup table approach halves the read operations for accessing the lookup table compared with GPU_{LUT} ,

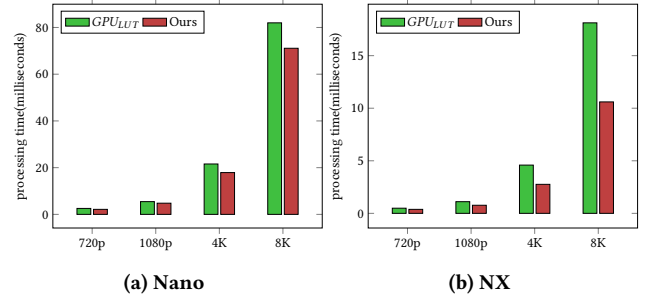


Figure 2: This graph shows computation performance of GPU_{LUT} and ours on the each devices.

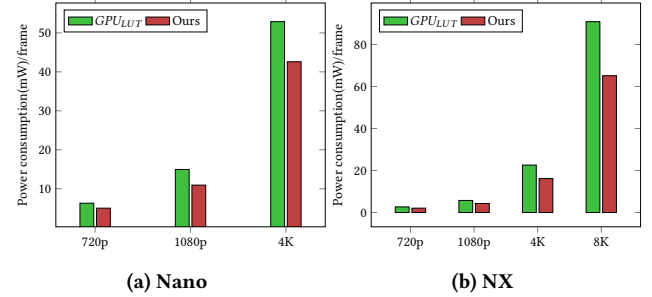


Figure 3: This graph compare the power consumption of GPU_{LUT} and our method on each edge devices for various resolution.

and it improves not only the processing performance but also power efficiency. Overall, our method reduces power usage up to 28.93% (24.86% on average). These results demonstrate the advantage of our method over the mapping approach on power efficiency.

ACKNOWLEDGMENTS

This work was supported by Institute of Information & communications Technology Planning & Evaluation (IITP) grant funded by the Korea government (MSIT) (No.2020-0-00594, Morphable Haptic Controller for Manipulating VR·AR Contents, 100%).

REFERENCES

- Nikolaos Bellas, Sek M Chai, Malcolm Dwyer, and Dan Linzmeier. 2009. Real-time fisheye lens distortion correction using automatically generated streaming accelerators. In *2009 17th IEEE Symposium on Field Programmable Custom Computing Machines*. IEEE, 149–156.
- John Cheng, Max Grossman, and Ty McKercher. 2014. *Professional CUDA C programming*. John Wiley & Sons. 136–199 pages.
- Konstantis Daloukas, Christos D Antonopoulos, Nikolaos Bellas, and Sek M Chai. 2010. Fisheye lens distortion correction on multicore and hardware accelerator platforms. In *2010 IEEE International Symposium on Parallel & Distributed Processing (IPDPS)*. IEEE, 1–10.
- Shehrzad Qureshi. 2013. Computer Vision Acceleration Using GPUs. http://developer.amd.com/wordpress/media/2013/06/2162_final.pdf. AMD Developer Central, Accessed: 2021-07-08.
- Warren Robinett and Jannick P Rolland. 1992. A computational model for the stereoscopic optics of a head-mounted display. *Presence: Teleoperators & Virtual Environments* 1, 1 (1992), 45–62.
- Sam Van der Jeught, Jan AN Buytaert, and Joris JJ Dirckx. 2012. Real-time geometric lens distortion correction using a graphics processing unit. *Optical Engineering* 51, 2 (2012), 027002.